

Report

Name: Lokesh Dhake UCI ID: Idhake

Performance Analysis

Cache Efficiency & Speed Looking at the simulation results, the system maintained a steady Cache Hit Rate of about 63–64% across all four runs. This confirms that the Cache-Aside pattern is doing its job: nearly two-thirds of the incoming read requests are being served directly from memory instead of hitting the slower MachineStateTable. This drastically improves performance because grabbing data from the cache is computationally much cheaper than simulating a database I/O operation.

Behavior Under Load Even with a solid cache hit rate, the Hit/Access ratio hovered around 0.57. This makes sense given the system's requirement for Strong Consistency. Since every POST request (like reserving or starting a machine) forces a direct write to the database to ensure accuracy, we expect the DB to still see significant traffic. Essentially, the system is balanced: the cache handles the bulk of the repetitive read traffic to prevent "read storms," while the database stays busy acting as the reliable source of truth for every state change.

Resource Usage The resource breakdown clearly shows the benefits of this caching strategy. The MachineStateTable only consumed about 4% of the total processing units, while the IdentityProviderClient used roughly 94%. This is a great outcome because it proves the database is no longer a bottleneck. The system's throughput is now mostly limited by the external authentication service, which is exactly what we want to see in a read-heavy microservice.

SnapShot:

```
loki@DESKTOP-6RV2R4L:/mnt/e/UCI Fall 25 Course Work/Advance Programming and Problem Solving/Systemdesign assignment/Systemdesign component implementation/cs-253p-hw-hampr-machine-service$ npm test
```

```
> hampr-base@1.0.0 test
> jest
```

```
PASS test/api.test.ts
PASS test/simulation.test.ts
  ● Console
```

console.log

(index)	Resource	Run 1 Units	Run 1 %	Run 2 Units	Run 2 %	Run 3 Units	Run 3 %	Run 4 Units	Run 4 %
0	'IdentityProviderClient'	1280512	'93.89%'	1280512	'93.77%'	1280512	'93.82%'	1280512	'93.74%'
1	'SmartMachineClient'	6912	'0.51%'	6912	'0.51%'	6912	'0.51%'	6912	'0.51%'
2	'MachineStateTable'	55112	'4.04%'	55112	'4.04%'	55112	'4.04%'	55112	'4.03%'
3	'DataCache'	23546	'1.73%'	23546	'1.72%'	23546	'1.73%'	23546	'1.72%'

at Object.<anonymous> (test/simulation.test.ts:160:13)

console.log

(index)	Run	Cache Hits	Cache Misses	Hit Rate
0	1	3819	2085	'64.68%'
1	2	3741	2196	'63.01%'
2	3	3800	2163	'63.73%'
3	4	3700	2209	'62.62%'

at Object.<anonymous> (test/simulation.test.ts:161:13)

console.log

(index)	Run	Cache Hits	DB Accesses	Hit/Access Ratio
0	1	3819	6600	'0.5786'
1	2	3741	6600	'0.5668'
2	3	3800	6600	'0.5758'
3	4	3700	6600	'0.5606'

at Object.<anonymous> (test/simulation.test.ts:162:13)

```
Test Suites: 2 passed, 2 total
Tests: 12 passed, 12 total
Snapshots: 0 total
Time: 6.758 s, estimated 7 s
```

```
loki@DESKTOP-6RV2R4L:/mnt/e/UCI Fall 25 Course Work/Advance Programming and Problem Solving/Systemdesign assignment/Systemdesign component implementation/cs-253p-hw-hampr-machine-service$
```